Поисково-аналитическая NoSQL-система по студентам университета на OpenSearch

Итоговый отчёт — Модуль 3 «Нереляционные базы данных»

Орешко Владислав • 18.03.2026

1. Схема базы данных

Общая концепция

Проект реализует поисково-аналитическую систему на базе OpenSearch для информационной системы университета. Данные хранятся в двух индексах: **students** — профили студентов с вложенным массивом дисциплин — и **student_activity** — журнал событий с высокой частотой записи, фиксирующий каждое взаимодействие студента с цифровыми сервисами вуза.

Индекс students (~5 000 документов)

Один документ — один студент. Массив курсов хранится как тип **nested**, что позволяет выполнять запросы на уровне курса без объединения таблиц. Поле **gpa** — тип **float**, что обеспечивает поддержку агрегации **avg** напрямую в OpenSearch. Поля-идентификаторы имеют тип **keyword** для точного совпадения при агрегациях.

Поле	Тип	Описание
student_id	keyword	Уникальный ID студента, напр. S000672
full_name	text + keyword	Полное имя: полнотекстовый поиск + сортировка
faculty	keyword	Факультет — используется в агрегациях
program	keyword	Образовательная программа
year	integer	Год обучения (1–4)
age	integer	Возраст студента
gpa	float	Средний балл (6–10); индексируется для avg
city	keyword	Родной город
dormitory	boolean	Проживание в общежитии
created_at	date	Дата создания записи
tags	keyword[]	Свободные теги: scholarship, olympiad и др.
courses	nested	Массив записанных дисциплин (см. ниже)

Вложенный тип courses

Поле	Тип	Описание
course_id	keyword	Код курса, напр. DB101, ML301
course_name	text + keyword	Название курса

grade	integer	Оценка (6–10)

Индекс student_activity (~12 000–25 000 документов)

Каждый документ — одно событие. Поле **details** — тип **object**, содержит ip, duration_ms и result. Поле **course_id** позволяет связать событие с конкретной дисциплиной без использования JOIN.

Поле	Тип	Описание
event_id	keyword	Уникальный ID события, напр. E00022208
student_id	keyword	Ссылка на студента
event_type	keyword	Тип: login, submit_assignment, library_visit и др.
source	keyword	Источник: python_cli, web_portal, mobile_app, admin_panel
course_id	keyword	Связанный курс
timestamp	date	Метка времени события
details	object	Вложенный объект: ip, duration_ms, result

ER-диаграмма

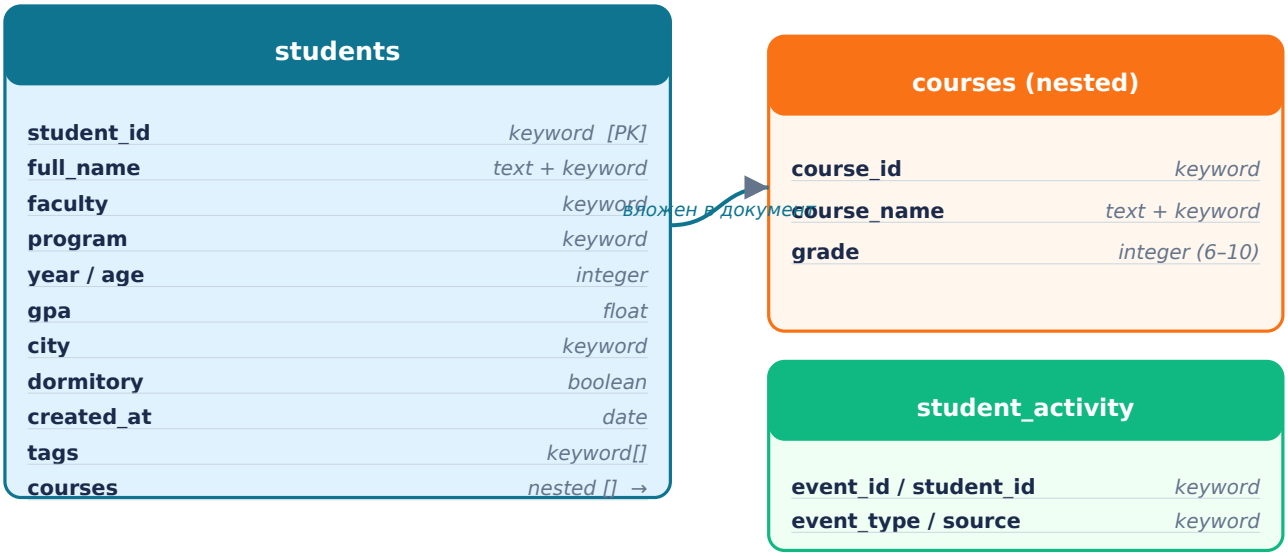
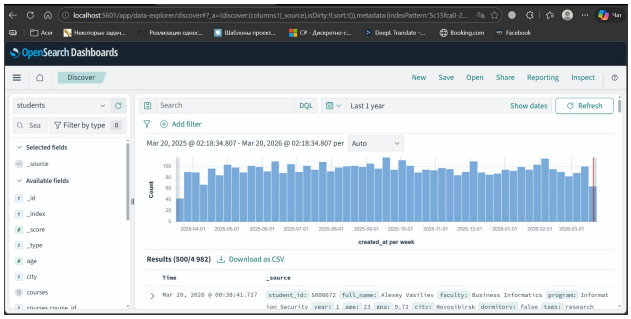


Рис. 1: Схема индексов. students содержит вложенный тип courses; student_activity — независимый индекс событий со ссылкой на student_id.



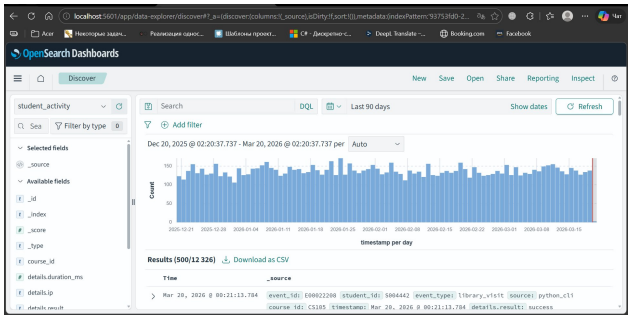


Рис. 2 (слева): индекс students — 4 982 документа, поле courses с вложенными объектами. Рис. 3 (справа): индекс student_activity — 12 326 событий, равномерный поток за 90 дней.

Реализация шардинга

Трёхузловой кластер OpenSearch с репликацией шардов

2. Архитектура шардинга

Обоснование выбора

OpenSearch реализует шардинг как нативную часть своей распределённой архитектуры: каждый индекс разбивается на **первичные шарды**, каждый из которых хранит подмножество документов. Дополнительно для каждого первичного шарда создаётся одна или несколько **реплик** на других узлах — для отказоустойчивости и параллельного чтения.

В данном проекте выбрана стратегия **горизонтального шардинга на уровне документов** (стандартная для OpenSearch): маршрутизация осуществляется по хешу поля **_id**, что обеспечивает равномерное распределение данных без ручного задания ключей шардирования. Этот подход идеально подходит для агрегаций (средний GPA, количество студентов по факультетам), которые OpenSearch автоматически выполняет в режиме **scatter-gather** по всем шардам.

Конфигурация кластера

Параметр	Значение	Обоснование
Количество узлов	3	Нечётное число — защита от split-brain при выборе лидера
Первичных шардов	3	По одному на узел — максимальная параллельность записи
Реплик на шард	1	Каждый первичный шард продублирован на другом узле
Итого шардов	6 (students)	3 первичных + 3 реплики
Стратегия маршрутизации	hash(_id) → шард	Автоматическое равномерное распределение по _id
Статус кластера	GREEN	Все 26 шардов активны, 0 неназначенных
Объём кучи (JVM)	1 GB на узел	Достаточно для датасета ~5K + 25K документов
Порты	9201/9202/9203	node1 — основной; Dashboards — 5602

Распределение шардов по узлам

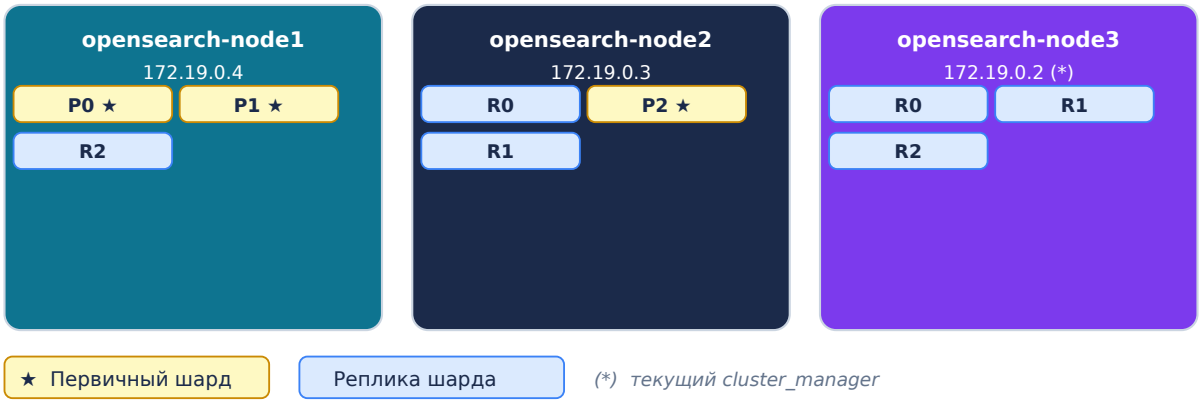
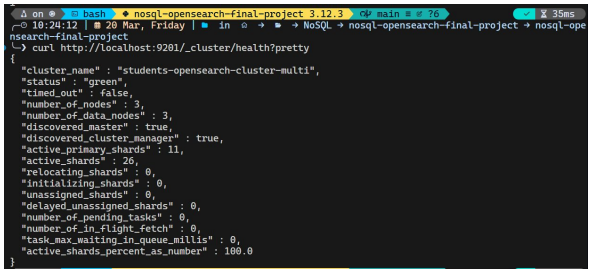


Рис. 4: Диаграмма шардов. Жёлтый — первичный шард (★), синий — реплика. Каждый узел содержит шарды разных ролей, что исключает потерю данных при отказе одного узла.



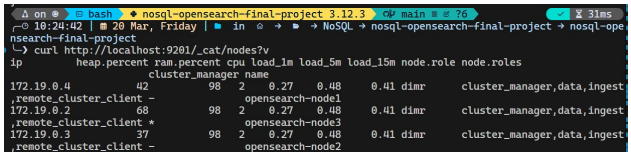


Рис. 5 (слева): health кластера — статус GREEN, 11 первичных шардов активны, 0 неназначенных. Рис. 6 (справа): все 3 узла запущены; node3 выбран cluster_manager (*).

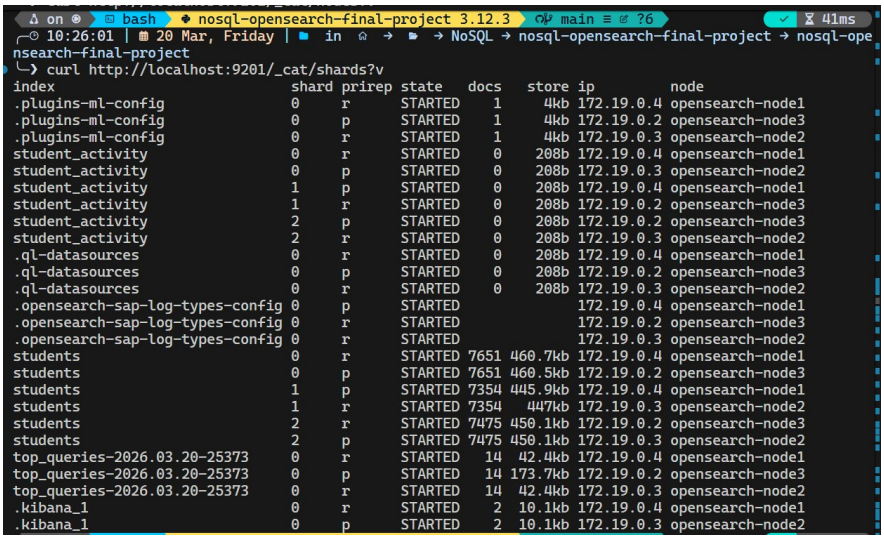


Рис. 7: Вывод /_cat/shards?v — все шарды индекса students (P + R) в статусе STARTED, каждый из трёх шардов занимает ~445–461 КБ и распределён по всем узлам кластера.

Нагрузочное тестирование

Кластер vs. одиночный узел — массовая вставка и поисковые запросы

6 типов запросов • 100 итераций каждый

3. Анализ производительности

Методология

Нагрузочное тестирование **single-node** конфигурации выполнялось с помощью **OpenSearch Benchmark**. Для **cluster**-конфигурации тот же инструмент в локальном окружении WSL2 + Docker Compose упирался в ограничения собственного in-memory metrics store, из-за чего итоговые summary-метрики формировались некорректно. Поэтому для сравнительного анализа single-node и cluster были дополнительно выполнены повторяемые прикладные замеры времени выполнения реальных запросов к OpenSearch с помощью скрипта **scripts/benchmark.py**.

Скрипт поочерёдно запускался против **одиночного узла** (порт 9200) и **трёхузлового кластера** (порт 9201). Для каждого окружения:

1. Создаются временные индексы с соответствующими маппингами (1 шард для single-node, 3 шарда для кластера).
2. Выполняется **массовая вставка** (bulk insert) батчами по 1 000 документов, замеряется пропускная способность (docs/sec).
3. Выполняются **6 поисковых запросов** по 100 повторений каждый, фиксируется средняя задержка, медиана, p95 и p99 (мс), а также пропускная способность (ops/sec).

Тип запроса	Индекс	Описание
term_faculty	students	Точный поиск по полю faculty
term_program	students	Точный поиск по полю program
nested_course	students	Вложенный запрос по courses.course_id
agg_faculty	students	Terms-агрегация: студентов по факультетам
avg_gpa	students	Метрическая агрегация: средний GPA
activity_by_source	student_activity	Terms-агрегация: событий по источнику

Результаты: поисковые запросы

Средняя задержка поиска (мс) — меньше лучше

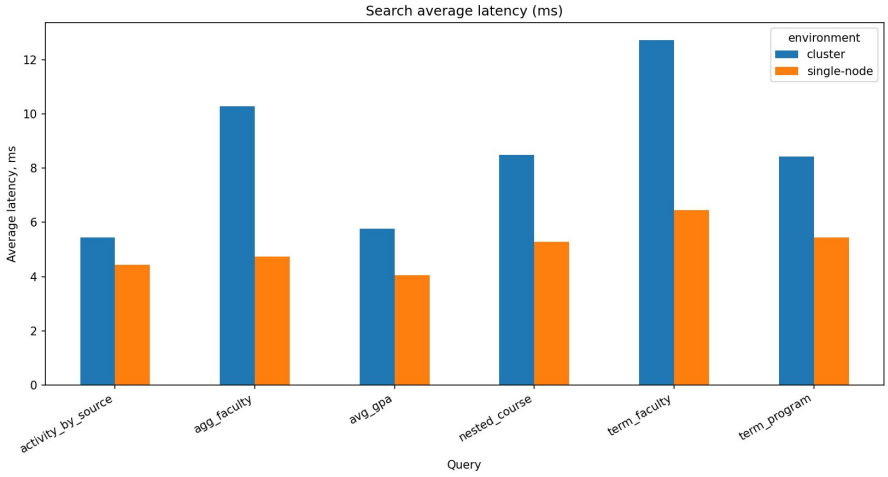
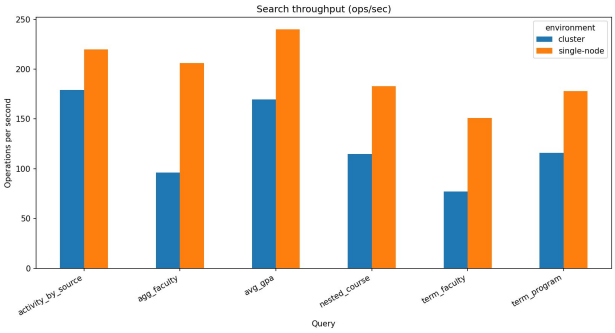


Рис. 8: Средняя задержка поиска (мс). Кластер (синий) медленнее на 20–100% из-за накладных расходов на межузловую координацию при данном масштабе данных.



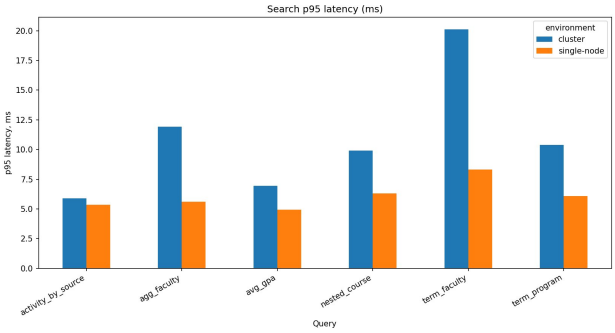


Рис. 9 (слева): Пропускная способность поиска — одиночный узел достигает 150–240 ops/sec против 75–180 у кластера. Рис. 10 (справа): P95-задержка — «хвостовые» задержки кластера до 2.5× выше; наибольший разрыв у term_faculty: 20 мс (кластер) против 8.3 мс (одиночный узел).

Результаты: массовая вставка

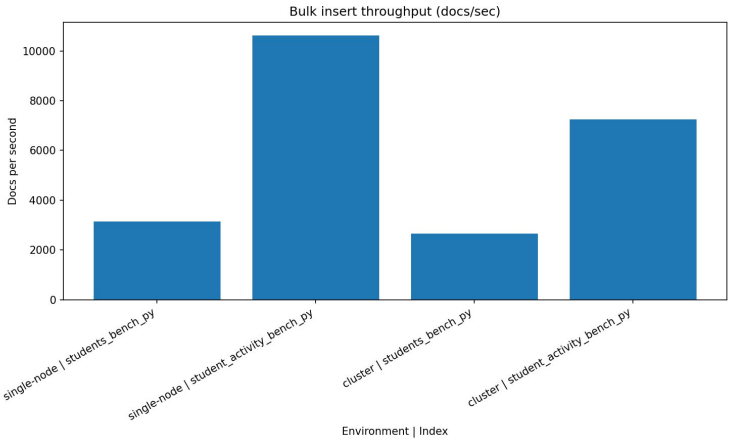


Рис. 11: Пропускная способность bulk insert (docs/sec). Документы student_activity значительно проще по структуре, что объясняет пропускную способность в 3–4 раза выше. Кластер уступает одиночному узлу на ~15% для students и на ~30% для student_activity — запись в реплику добавляет сетевые задержки.

Сводная таблица и выводы

Метрика	Кластер	Одиночный узел	Комментарий
Avg задержка (мс)	5.5–12.7	4.0–6.4	Кластер медленнее из-за сетевых round-trip
P95 задержка (мс)	5.9–20.1	4.9–8.3	«Хвост» у кластера в 1.5–2.5 раза выше
Пропускная способность	75–180 ops/s	150–240 ops/s	Одиночный узел выигрывает при малом объёме
Вставка students	~2 661 docs/s	~3 139 docs/s	Репликация добавляет ~15% накладных расходов
Вставка activity	~7 244 docs/s	~10 617 docs/s	Простые документы, разница ощутимее

В рамках проведённого эксперимента кластер OpenSearch показал более низкую производительность по сравнению с single-node конфигурацией как при записи, так и при выполнении поисковых запросов. Это объясняется следующими факторами:

- накладные расходы на сетевое взаимодействие между нодами;
- необходимость координации запросов (scatter-gather);
- репликация данных при записи.

Однако при увеличении объёма данных и нагрузки кластерная архитектура обеспечивает:

- горизонтальное масштабирование;
- устойчивость к отказам;
- более высокую пропускную способность при больших нагрузках.

Таким образом, использование кластера оправдано в production-сценариях с большими объёмами данных, тогда как single-node эффективнее для небольших систем и локальной разработки.